AI for Physics Projects

Setup instructions for macOS, Windows and Linux

Please complete *before* the course on **21.10.2026**

Why this matters

The hands-on session lasts one hour. If we spend it installing software, we lose it. Plan for **45–60 minutes**, most of it unattended while Julia packages download and compile. If anything fails, find me in the corridor before the course; do not wait until the day.

Contents

1 What you will install, and why **2**

2 Accounts **3**

2.1 GitHub . . . 3

2.2 An AI coding agent . . . 3

3 macOS **3**

3.1 Homebrew (package manager) . . . 3

3.2 Git, GitHub CLI, VS Code, GitHub Desktop . . . 3

3.3 Julia . . . 4

4 Windows **4**

4.1 Git . . . 4

4.2 GitHub CLI, VS Code, GitHub Desktop . . . 4

4.3 Julia . . . 4

5 Linux **5**

5.1 Git and curl . . . 5

5.2 GitHub CLI . . . 5

5.3 VS Code . . . 5

5.4 Julia . . . 5

6 Configuration (all systems) **6**

6.1 Git . . . 6

6.2 Connect Git to GitHub (SSH) . . . 6

6.3 VS Code 7
6.4 Course material and Julia packages 7
6.5 AI agent: one-minute test 8

7 Final check **9**

8 Troubleshooting **9**

1 What you will install, and why

Tool	Required?	Role in the course
Git	required	Version control: the undo button and lab notebook that make it safe to let an agent edit your files.
GitHub account	required	Hosts repositories; runs automated tests (continuous integration, CI).
GitHub CLI (`gh`)	required	Logs you into GitHub and sets up an SSH key for Git in one step.
Julia (via `juliaup`)	required	The language of all examples.
Julia packages	required	DrWatson (project structure), QuantumOptics, OrdinaryDiffEq, LsqFit, CairoMakie, JLD2, Revise.
VS Code + Julia extension	required	Editor; also the easiest way to *read diffs*, a central skill of the course.
AI coding agent	required	An LLM tool that works directly in a folder on your computer (e.g. Claude Code or Codex). See Section 2.2.
Course repository	required	Slides, exercises and the hands-on project; you clone it at the end of the setup.
GitHub Desktop	optional	Graphical Git client (macOS and Windows only).

Requirements.

- About **6 GB** of free disk space (Julia packages are the bulk of it).
- Permission to install software. On a university-managed laptop you may need IT support; ask *now*, not the day before.
- A working internet connection without a restrictive proxy. If your institute uses one, see Section 8.

Order. Do Section 2 (accounts), then *only* the section for your operating system (3, 4 or 5), then Sections 6–7, which are the same for everyone.

Terminal. Several steps use a terminal:

- **macOS:** *Terminal* (Applications → Utilities).
- **Windows:** *Terminal* or *PowerShell* from the Start menu. Do *not* use the old *Command Prompt* (`cmd.exe`).
- **Linux:** your usual terminal emulator.

Type or paste each command, then press Enter. Lines starting with `#` are comments; you do not need to type them.

2 Accounts

2.1 GitHub

1. Create a free account at https://github.com/signup. Use an email address you will keep; your university address also qualifies you for free extras via https://education.github.com.
2. Note your **username** and the **email** you registered with; you will need both in Section 6.1.

2.2 An AI coding agent

You need access to an LLM with *local agent* capabilities: a tool that runs on your computer, inside a project folder, and can read and edit files and run commands (with your permission). A chat window in the browser is *not* enough. Examples are **Claude Code** (Anthropic) and **Codex** (OpenAI); use whichever matches the subscription you already have.

Install it following its own instructions, and make sure that before the course you can (i) start it in a folder and (ii) log in. Section 6.5 has a one-minute test. The course does not depend on which agent you use.

3 macOS

Works on Apple Silicon (M1 and later) and Intel Macs, macOS 13 or newer.

3.1 Homebrew (package manager)

Homebrew installs and updates developer tools with one command each. Check whether you already have it:

```
brew --version
```

If this prints a version, skip to Section 3.2. Otherwise install it:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/
    install.sh)"
```

It will ask for your Mac password (nothing appears as you type; that is normal) and may install the *Xcode Command Line Tools*, which takes a few minutes.

Apple Silicon: read the last lines of the output

At the end, Homebrew prints "Next steps" with two or three commands that add `brew` to your `PATH` (they contain `/opt/homebrew/bin/brew shellenv`). **Copy and run them exactly as printed**, then close and reopen Terminal.

3.2 Git, GitHub CLI, VS Code, GitHub Desktop

```
brew install git gh
brew install --cask visual-studio-code
brew install --cask github          # optional: GitHub Desktop
```

macOS ships an older Git with the Xcode tools; the Homebrew version takes precedence once `brew` is on your `PATH`.

3.3 Julia

Install `juliaup`, the official Julia version manager:

```
curl -fsSL https://install.julialang.org | sh
```

Accept the defaults (press Enter). **Close and reopen Terminal**, then check:

```
julia --version
```

Do not install Julia through `brew install julia` or from a `.dmg`; use `juliaup` so everyone has the same setup. (`brew install juliaup` is also fine, but install `juliaup` only one way, not both.)

Continue with Section 6.

4 Windows

Windows 10 (version 1809 or later) or Windows 11. All commands below run in **PowerShell** or **Windows Terminal**, not in Command Prompt. We use `winget`, the package manager built into Windows. Check it works:

```
winget --version
```

If the command is not found, install *App Installer* from the Microsoft Store and reopen the terminal.

4.1 Git

```
winget install --id Git.Git -e --source winget
```

This installs *Git for Windows*, which includes *Git Bash*. Several AI agents use Git Bash on Windows, so do not skip this step even if you plan to use GitHub Desktop. If you install Git from `https://git-scm.com` instead, the default options are fine; in particular keep *"Git from the command line and also from 3rd-party software"* and *"Checkout Windows-style, commit Unix-style line endings"*.

4.2 GitHub CLI, VS Code, GitHub Desktop

```
winget install --id GitHub.cli -e
winget install --id Microsoft.VisualStudioCode -e
winget install --id GitHub.GitHubDesktop -e     # optional
```

4.3 Julia

```
winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

This installs `juliaup` together with the current stable Julia. If the Microsoft Store is blocked on your machine, download the installer from https://julialang.org/downloads/ instead.

Reopen the terminal

After installing, **close all terminal windows and open a new one**; otherwise the new commands are not found. Then check:

```
git --version
gh --version
code --version
julia --version
```

Continue with Section 6.

5 Linux

Commands are given for Ubuntu/Debian, Fedora and Arch. Use the line for your distribution.

5.1 Git and curl

```
sudo apt update && sudo apt install -y git curl     # Ubuntu / Debian
sudo dnf install -y git curl                        # Fedora
sudo pacman -S --needed git curl                    # Arch
```

5.2 GitHub CLI

```
sudo dnf install -y gh                              # Fedora
sudo pacman -S github-cli                           # Arch
```

On **Ubuntu/Debian**, the version in the default repositories is often outdated. Follow the official instructions (one copy-paste block) at https://github.com/cli/cli/blob/trunk/docs/install_linux.md.

5.3 VS Code

- **Ubuntu/Debian:** download the `.deb` file from https://code.visualstudio.com/download, then

  ```
  sudo apt install ./code_*.deb
  ```

- **Fedora:** download the `.rpm` file from the same page, then `sudo dnf install ./code-*.rpm`.
- **Arch:** `visual-studio-code-bin` from the AUR.

These packages add Microsoft's repository, so VS Code updates with your system. GitHub Desktop has no official Linux version; use VS Code's built-in Git view instead.

5.4 Julia

```
curl -fsSL https://install.julialang.org | sh
```

Accept the defaults, **close and reopen the terminal**, and check `julia -version`. Do not use the `julia` package from your distribution; it is usually outdated or built differently.

Continue with Section 6.

6 Configuration (all systems)

6.1 Git

Tell Git who you are. Use the **same email as your GitHub account**, otherwise your commits will not be linked to you.

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global pull.rebase false
```

The last three are recommended defaults: new repositories start on a branch called `main`, matching GitHub; when Git needs you to write a message it opens VS Code instead of `vim`; and `git pull` merges instead of stopping with "*Need to specify how to reconcile divergent branches*". Line endings (one command, depending on your system):

```
git config --global core.autocrlf input     # macOS and Linux
git config --global core.autocrlf true      # Windows
```

Check your settings with `git config -global -list`.

6.2 Connect Git to GitHub (SSH)

In this course Git talks to GitHub over **SSH**: your computer holds a private key, GitHub holds the matching public key, and no password is ever typed. The GitHub CLI creates the key and registers it with GitHub in one go:

```
gh auth login
```

Answer the questions as follows:

1. Where do you use GitHub? → **GitHub.com**
2. Preferred protocol for Git operations? → **SSH**
3. Generate a new SSH key to add to your GitHub account? → **Yes**
   (If you already have a key, it asks instead whether to *upload* your existing public key; choose that key.)
4. Enter a passphrase for your new SSH key → press **Enter** to leave it empty, unless you already know how to use an SSH agent (see the box below).
5. Title for your SSH key → accept the default, or type the name of this computer.
6. How would you like to authenticate? → **Login with a web browser**

Copy the one-time code it shows, press Enter, paste the code in the browser and approve. Then check both the GitHub login and the SSH connection:

```
gh auth status
ssh -T git@github.com
```

`gh auth status` should say *"Logged in to github.com account ..."* and *"Git operations protocol: ssh"*. The first time you run `ssh -T`, it asks *"Are you sure you want to continue connecting?"*: type **yes**. It should then answer *"Hi <username>! You've successfully authenticated, but GitHub does not provide shell access."* That message means it worked.

Passphrase or not?

The key file is what proves you are you, so anyone who copies it can push as you. A passphrase protects it, but you then have to unlock it once per session. On **macOS** this is painless: add these lines to the file `~/.ssh/config` (create it if it does not exist) and macOS remembers the passphrase in its keychain:

```
Host github.com
  AddKeysToAgent yes
  UseKeychain yes
```

On Windows and Linux, a passphrase requires running an SSH agent; if you have never done this, leave the passphrase empty for now.

Cloning. Always copy the **SSH** address of a repository (`git@github.com:user/repo.git`), not the HTTPS one: on GitHub, click the green *Code* button and choose the *SSH* tab. `gh repo clone user/repo` uses SSH automatically.

6.3 VS Code

1. **macOS only:** open VS Code, press `Cmd+Shift+P`, type *shell command*, and choose *"Shell Command: Install 'code' command in PATH"*. (Windows and Linux do this automatically.)
2. Install the Julia extension from a terminal:

   ```
   code --install-extension julialang.language-julia
   ```

   Many AI agents also offer a VS Code extension; installing it is optional.
3. Open VS Code once and click the *Source Control* icon in the left bar. This is where we will read diffs during the course.

6.4 Course material and Julia packages

This step copies the course repository to your computer (this also tests your SSH connection), then downloads and compiles everything the hands-on session needs. It takes **10–20 minutes**; start it and let it run. Create a folder for the course in your home folder and clone the repository into it:

```
cd ~
mkdir ai-course
cd ai-course
git clone git@github.com:marcelojbp/physics_RSE_and_AI.git course
cd course
julia setup/setup_julia.jl
```

The script ends with *"All good: Julia ... is ready for the course."* It installs the packages in the table below, then the exact package versions of the hands-on project `DrivenSpin`, and runs that project's tests.

Keep the folder `ai-course`: we start from it on the day, and update the course material with `git pull` inside `course`. Do not edit files inside `course`; in the hands-on you work on your own copy.

Keep this Julia version until the course

Packages are compiled for one Julia version. If you update Julia (e.g. with `juliaup update`) after this step, everything is compiled again on the day.

If cloning fails (and Section 8 does not help), still prepare the packages: start Julia by typing `julia` in a terminal, then paste the following at the `julia>` prompt, and tell me about the clone error before the course.

```
using Pkg
Pkg.add(["DrWatson", "Revise"])
Pkg.activate(temp=true)
Pkg.add(["DrWatson", "QuantumOptics", "OrdinaryDiffEq",
         "LsqFit", "CairoMakie", "JLD2"])
Pkg.precompile()
using DrWatson, QuantumOptics, OrdinaryDiffEq, LsqFit, CairoMakie, JLD2
println("All packages loaded.")
```

Leave Julia with `exit()`.

What the packages are for:

DrWatson	Project structure, reproducible file names, saving results tagged with the Git commit.
QuantumOptics	Reference solver for spins and cavities; used as an independent check.
OrdinaryDiffEq	Time evolution (the ODE part of DifferentialEquations.jl, lighter to install).
LsqFit	Fitting synthetic "experimental" data.
CairoMakie	Plotting.
JLD2	File format DrWatson uses to save results.
Revise	Reloads your code automatically as you (or the agent) edit it.

`Test` and `LinearAlgebra` ship with Julia and need no installation.

DrWatson and Revise go into your default environment, so they are always available. The other packages are downloaded into Julia's shared cache; your copy of the hands-on project reuses them, so you will not wait for compilation on the day.

6.5 AI agent: one-minute test

Agents work inside a folder, so create an empty practice folder and start your agent there (from the terminal or its app, depending on the tool):

```
mkdir ai-course-test
cd ai-course-test
```

Log in if asked, then give it a first request:

```
Write a Julia script hello.jl that prints the Julia version, and run it.
```

The test passes if the agent creates `hello.jl` in that folder, runs it with *your* Julia, and shows the version printed by `julia -version`. It will usually ask permission before creating the file and before running it; approve both. In the course the agent also runs Git, so give it a second request:

```
Make this folder a git repository and commit hello.jl. Then show me git log.
```

The log should show one commit with your name. You can delete the folder afterwards.

7 Final check

In a **new** terminal, run each of these. Every line should print a version or "logged in" and no error.

```
git --version
git config user.email
gh auth status
ssh -T git@github.com
code --version
julia --version
julia -e "using DrWatson"
cd ~/ai-course/course
git status
```

`ssh -T` should greet you by your GitHub username (see Section 6.2). `julia -e "using DrWatson"` prints nothing when it works and an error when it does not. `git status` should say *"On branch main. Your branch is up to date with 'origin/main'."* Then tick the list:

- ☐ I have a GitHub account and `gh auth status` says I am logged in, with protocol `ssh`.
- ☐ `ssh -T git@github.com` answers "Hi <username>! You've successfully authenticated…".
- ☐ `git config user.email` shows the email of my GitHub account.
- ☐ VS Code opens and has the Julia extension.
- ☐ The course repository is in `ai-course/course`, and `julia setup/setup_julia.jl` finished with "All good".
- ☐ My AI agent starts in a folder, I am logged in, and it created, ran and committed `hello.jl`.

Bring your laptop **and its charger** on **21.10.2026**.

8 Troubleshooting

"command not found" / "is not recognized" right after installing
Close *all* terminal windows and open a new one. On Windows, if that fails, sign out and back in. On macOS, check that the Homebrew "Next steps" commands were run.

Julia package installation stalls or fails with network errors
Usually a proxy or firewall. Try from home or eduroam. If your institute requires a proxy, set it before starting Julia (ask IT for the address):

```
export HTTPS_PROXY=http://proxy.example.org:8080     # macOS / Linux
$env:HTTPS_PROXY="http://proxy.example.org:8080"     # Windows PowerShell
```

Then run the installation again; it resumes where it stopped.

Precompilation is very slow
Expected on the first run, especially for CairoMakie and OrdinaryDiffEq. Keep the laptop plugged in and awake. It only happens once.

The agent cannot find `julia` or `git`
Start the agent from a *new* terminal opened after installing everything, so it inherits the updated `PATH`.

`ssh -T git@github.com` says "Permission denied (publickey)"
Your key is missing or not registered with GitHub. Run `gh ssh-key list` to see the keys GitHub knows about. If the list is empty, run `gh auth refresh -s admin:public_key` and then `gh ssh-key add ~/.ssh/id_ed25519.pub`.

`ssh -T` hangs or says "Connection timed out"
Many institute networks block SSH's usual port (22). GitHub also accepts SSH on port 443. Add these lines to `~/.ssh/config` (create the file if needed; on Windows it is `.ssh\config` in your user folder):

```
Host github.com
  Hostname ssh.github.com
  Port 443
  User git
```

and run `ssh -T git@github.com` again. If you already have a `Host github.com` block (from the passphrase box), add the three lines to it.

`git clone` says "Repository not found" or "Could not read from remote repository"
Check the SSH connection first with `ssh -T git@github.com` (see the two items above). If that works, check that you copied the address exactly.

`git push` asks for a username or password
The repository was cloned with its HTTPS address. Switch it to SSH, from inside the repository folder:

```
git remote set-url origin git@github.com:<user>/<repo>.git
```

and make SSH the default for the GitHub CLI: `gh config set git_protocol ssh`.

University-managed laptop, no admin rights
`juliaup` installs into your user folder and does not need admin rights. Git, VS Code and the GitHub CLI usually do; ask IT, or on Windows use `winget install` with `-scope user` where available.

If you are stuck, note the *exact* error message and your operating system, and come to me before the course.